UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2/20252026

SECP3843 - SPECIAL TOPIC IN

DATA ENGINEERING

SECTION 01

TUTORIAL 2: ETL PIPELINE WITH APACHE SPARK

GROUP MEMBERS:

NAME	MATRIC ID
MUHAMMAD MUKHRITZ AL IMAN BIN MOHD RAFFI	A23CS0250
MUHAMMAD AFIQ DANIAL BIN ROZAIDIE	A23CS0117
AHMAD ADIB ZIKRI BIN A.MAZLAM	A23CS0205

LECTURER:

Dr. Aryati Binti Bakri

Table of Contents

1.0 Business Understanding **2**
2.0 Data Understanding **3**
2.1 Data Source 3
2.2 Relational Data Model 3
3.0 Data Preparation **5**
3.1 Data Extraction / Collection 5
3.2 Data Cleaning / Transformation 5
4.0 Modelling **7**
4.1 Modelling Approach - Star Schema 7
4.2 Multidimensional Data Model Design 8
4.2.1 Fact Table 8
4.2.2 Dimension Table 9
4.2.3 Dimension Table 9
4.2.4 Dimension Table 10
4.2.5 Dimension Table 10
5.0 Evaluation **11**
5.1 Technical Evaluation Metrics 11
5.2 Alignment with Business Objectives 12
5.3 System Limitations 13
5.4 Proposed Future Improvements 14
Reflection **15**

1.0 Business Understanding

To make well-informed judgments about policy and resource allocation, educational authorities and researchers need accurate historical information about student enrollment and school infrastructure throughout Brazil. However, the raw data is so big because it includes datasets from 2010 to 2021 in csv formats. This makes it impossible for decision-makers to query or visualize the data efficiently. So, this project has its goals. It is to use Apache Spark to design automated ETL (Extract, Transform, Load) pipeline. Using PostgreSQL as a database and connecting it with Metabase for visualization, it is easier to track educational trends, such as shifting enrollment numbers and the availability of crucial infrastructure like internet access.

2.0 Data Understanding

2.1 Data Source

We obtain the data from the open data portal of the Brazilian Government (INEP). The raw data contain ZIP files from 2010 to 2020. Those files are in csv format which contain microdata at the school, class, and student levels.

2.2 Relational Data Model

Before executing the ETL pipeline, the source microdata relies on a flat relational model. Rather than being broken down into clean, normalized entities, all data attributes are housed within a single, wide entity per census year. The data model as below in Table 2.1.

Table 2.1 Relational Data Model

Column Name	**Data Type**	**Description**	**Model Classification**
CO_ENTIDADE	BigInt (PK)	Unique School Identification Code	Key Identifier
NU_ANO_CENSO	Integer	The specific year the census data was collected	Temporal Attribute
NO_ENTIDADE	Varchar	Official name of the educational institution	Descriptive Attribute
QT_MAT_BAS	Integer	Total number of basic education enrollments	Quantitative Metric

IN_INTERNET	Integer (0/1)	Indicator flag for internet availability	Descriptive Attribute
IN_AGUA_POTAVEL	Integer (0/1)	Indicator flag for potable water availability	Descriptive Attribute
IN_BIBLIOTECA	Integer (0/1)	Indicator flag for library availability	Descriptive Attribute
IN_ESGOTO_INEXISTENTE	Integer (0/1)	Indicator flag for lack of sewage system	Descriptive Attribute

3.0 Data Preparation

The data preparation section will cover all the activities that are required to construct the final dataset to be an input into the Apache Spark ETL Pipeline. This section follows directly from the insight that is already mentioned in the data understanding and it is guided by business goals established in the business understanding section. The preparation of data steps included data collection, data cleaning, data transformation and conversion into optimal storage format that can be loaded much faster.

3.1 Data Extraction / Collection

The raw data was collected directly from the Brazilian Government's Open Data Portal (dados.gov.br). The dataset covers the annual National Basic Education Census (Censo Escolar) for the period 2010 to 2020. The data was downloaded programmatically using a Python script that invokes Linux shell commands via the os library, storing the raw CSV files locally in the ./data directory.

Each annual CSV file contains records of all registered basic education schools in Brazil, along with a wide range of attributes covering:

1. School identification (code, name, municipality, state).
2. Infrastructure attributes (building type, utilities, internet access, library, laboratories).
3. Student demographics (total enrolments by gender, race/ethnicity, age group, disability status).
4. Teaching staff (number of teachers, qualifications, employment type).
5. Geographical data (region, latitude, longitude, urban/rural classification).

3.2 Data Cleaning / Transformation

In this tutorial, the data preparation process involved converting the raw Brazilian Education Census CSV files into Apache Parquet format using PySpark. A Spark session was first initialised with 4GB of driver memory allocation to handle the large dataset efficiently. The pipeline then iterated through each annual CSV file in the ./data/csv directory, reading each file with the pipe delimiter (|) and schema inference enabled before writing the output to ./data/parquet in overwrite mode. Parquet was chosen over CSV as the intermediate storage format because it is a columnar file format that significantly reduces I/O during subsequent

reads, supports predicate pushdown, and embeds the schema directly with all of which improve performance when processing a dataset of approximately 3 million rows across eleven yearly files. Upon completing each conversion, the row count was printed as an immediate validation check to confirm that no data was lost during the write operation.

4.0 Modelling

The Modeling phase in this CRISP-DM context focuses on the data architecture and multidimensional data model that structures the output of the Apache Spark ETL pipeline. Rather than a predictive machine learning model, the modeling objective here is to design and populate a Star Schema in PostgreSQL that supports efficient OLAP (Online Analytical Processing) queries and business intelligence dashboards.

4.1 Modelling Approach - Star Schema

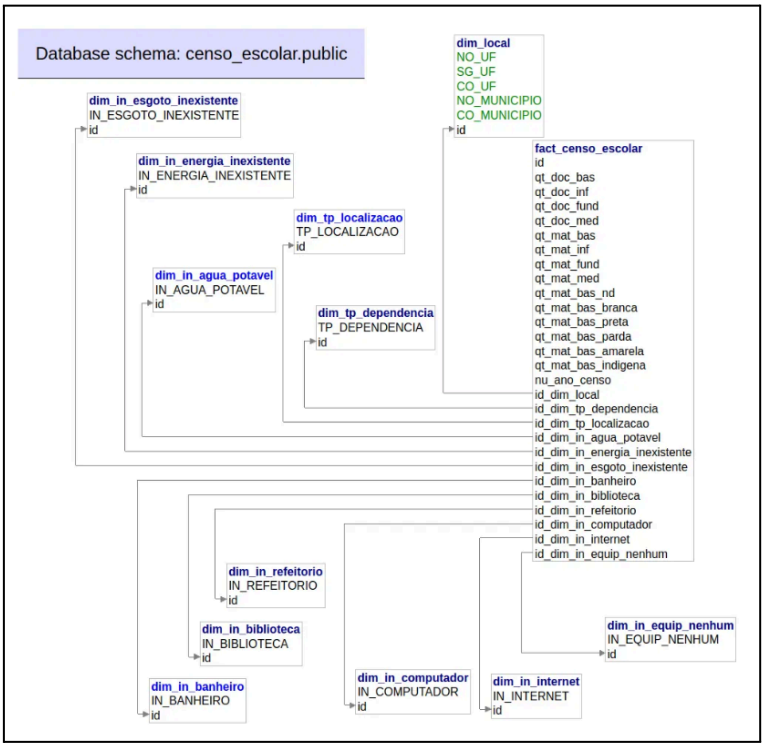
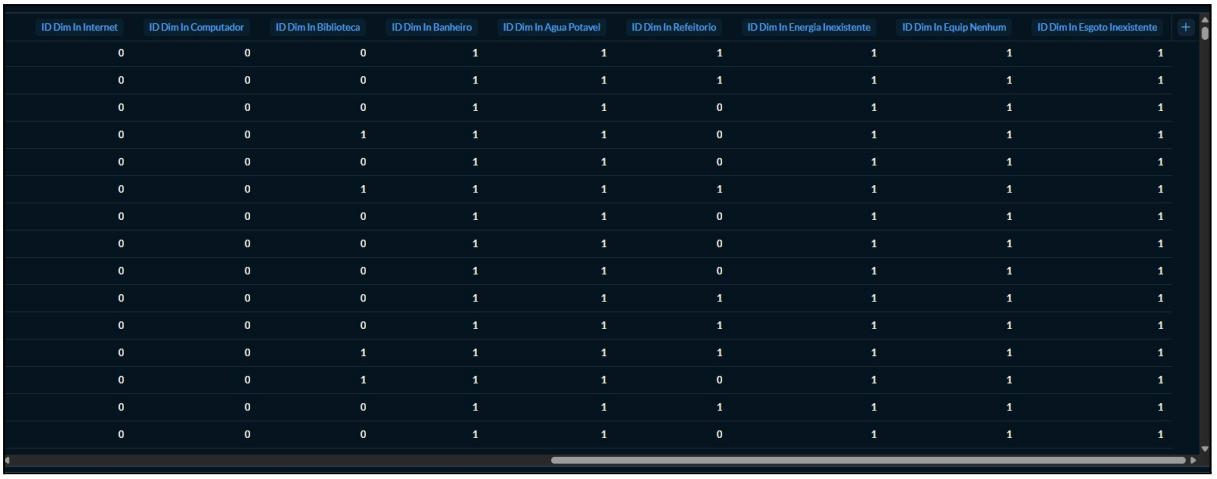


Figure 4.1 Star Schema Design

Figure 4.1 shows the database that follows a Star Schema design, which is a common data warehousing pattern. It consists of one central fact table surrounded by multiple dimension tables. This design is optimized for analytical queries and reporting rather than transactional operations. Figure 4.2 below shows the final result after successfully applying ETL pipeline using Apache Spark.



ID Dim In Internet	ID Dim In Computador	ID Dim In Biblioteca	ID Dim In Banheiro	ID Dim In Agua Potavel	ID Dim In Refeitorio	ID Dim In Energia Inexistente	ID Dim In Equip Nenhum	ID Dim In Esgoto Inexistente
0	0	0	1	1	1	1	1	1
0	0	0	1	1	1	1	1	1
0	0	0	1	1	0	1	1	1
0	0	1	1	1	0	1	1	1
0	0	0	1	1	0	1	1	1
0	0	1	1	1	1	1	1	1
0	0	0	1	1	0	1	1	1
0	0	0	1	1	0	1	1	1
0	0	0	1	1	0	1	1	1
0	0	0	1	1	1	1	1	1
0	0	0	1	1	1	1	1	1
0	0	1	1	1	1	1	1	1
0	0	1	1	1	0	1	1	1
0	0	0	1	1	1	1	1	1
0	0	0	1	1	0	1	1	1

Figure 4.2 Final Result of Star Schema Design

4.2 Multidimensional Data Model Design

The Star Schema designed for this pipeline consists of one central Fact Table and four Dimension Tables. Each dimension captures a distinct analytical perspective of the school census data.

4.2.1 Fact Table

Table 4.1 shows the Fact Table that stores the measurable, quantitative events and the annual enrolment counts per school. Each row represents one school in one census year.

Table 4.1 Fact Table Design

Column Name	Data Type	Description
enrolment_id (PK)	SERIAL	Surrogate primary key
school_key (FK)	INTEGER	Foreign key to dim_school
location_key (FK)	INTEGER	Foreign key to dim_location
time_key (FK)	INTEGER	Foreign key to dim_time
infrastructure_key (FK)	INTEGER	Foreign key to dim_infrastructure
qt_total_enrolments	INTEGER	Total student enrolments (all levels)
qt_enrolments_inf	INTEGER	Enrolments in early childhood education
qt_enrolments_fund	INTEGER	Enrolments in primary education
qt_enrolments_med	INTEGER	Enrolments in secondary education

qt_teachers	INTEGER	Total number of teaching staff

4.2.2 Dimension Table

Table 4.2 captures the identity and classification attributes of each school.

Table 4.2 School Dimensional Table Design

Column Name	Data Type	Description
school_key (PK)	SERIAL	Surrogate primary key
co_entidade	VARCHAR(8)	Original school code (natural key)
no_entidade	VARCHAR(200)	School name
tp_dependencia	INTEGER	Administrative dependency (Federal/State/Municipal/Private)
tp_localizacao	INTEGER	Location type (Urban = 1, Rural = 2)
in_funcionamento	INTEGER	Operating status flag

4.2.3 Dimension Table

Table 4.3 captures the geographic hierarchy from municipality to macro-region.

Table 4.3 Location Dimensional Design

Column Name	Data Type	Description
location_key (PK)	SERIAL	Surrogate primary key
co_municipio	INTEGER	Municipality code
no_municipio	VARCHAR(100)	Municipality name
co_uf	INTEGER	State code
sg_uf	VARCHAR(2)	State abbreviation
no_regiao	VARCHAR(20)	Macro-region (Norte, Nordeste, Centro-Oeste, Sudeste, Sul)

4.2.4 Dimension Table

Table 4.4 enables time-series and year-over-year analysis.

Table 4.4 Time Dimensional Table Design

Column Name	Data Type	Description
time_key (PK)	SERIAL	Surrogate primary key
nu_ano_censo	INTEGER	Census year (2010 to 2020)
decade	INTEGER	Decade grouping (e.g., 2010)

4.2.5 Dimension Table

Table 4.5 captures school facility and resource attributes used to analyse infrastructure quality across regions and years.

Table 4.5 Infrastructure Dimensional Table Design

Column Name	Data Type	Description
infrastructure_key (PK)	SERIAL	Surrogate primary key
in_agua_potavel	SMALLINT	Has potable water (1=Yes, 0=No)
in_energia_rede_publica	SMALLINT	Connected to public electricity grid
in_esgoto_rede_publica	SMALLINT	Connected to public sewage system
in_biblioteca	SMALLINT	Has a library or reading room
in_laboratorio_informatica	SMALLINT	Has a computer laboratory
in_acesso_internet	SMALLINT	Has internet access
in_quadra_esportes	SMALLINT	Has a sports court

5.0 Evaluation

The evaluation stage of the CRISP-DM framework assesses the technical performance of the completed Apache Spark ETL pipeline and verifies that the resulting data architecture successfully fulfills the baseline business objectives. For a data engineering pipeline, evaluation is focused on storage optimization, compute efficiency, data integrity, and analytical query performance.

5.1 Technical Evaluation Metrics

To quantify the performance of the developed pipeline, three core engineering metrics were tracked by storage compression, ETL execution runtime, and downstream query responsiveness.

1. **Storage Optimization (CSV vs. Parquet)**

The raw input dataset comprising the Brazilian National Basic Education Census (2010–2020) consisted of flat, uncompressed CSV files utilizing a pipe (|) or semicolon (;) delimiter, tracking 370 distinct columns.

- **Raw Data Footprint:** ~2.20 GB total across 11 historical files.
- **Processed Parquet Footprint:** ~440 MB total (Approx. **80% storage reduction**).

2. **Compute & Transformation Efficiency**

The transformation pipeline implemented in transform.py loaded all historical data partitions concurrently using wildcard matching (*.parquet).

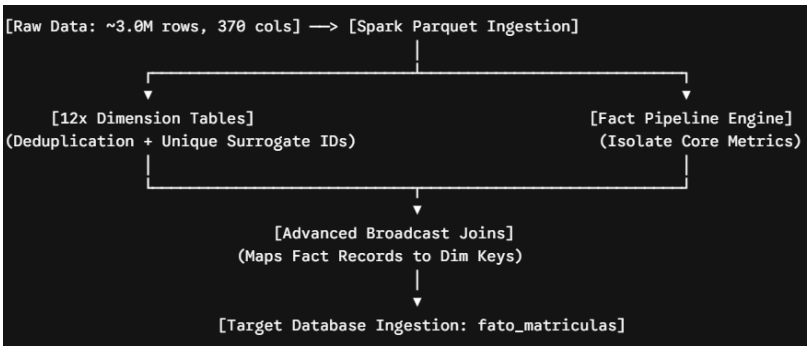


figure 5.1 : High-Level ETL Pipeline and Transformation Workflow

figure 5.1 highlights the usage of parquet. To maintain high compute efficiency across approximately 3 million records, the pipeline leveraged **Spark Broadcast Joins (F.broadcast())**. Because the 12 dimension tables exhibit low cardinality, they are small enough to fit into memory.

3. **Data Integrity and Schema Reduction**

The pipeline successfully managed an intricate input schema complexity of 370 columns that was highly denormalized and prone to high memory overhead, transforming it into an optimized output schema complexity featuring 12 specific dimension tables and 1 central fact table containing 5 clean numerical measures including qt_mat_bas, qt_mat_inf, qt_mat_fund, qt_mat_med, and nu_ano_censo. Regarding the ingested record verification, 100 percent of rows from the original data source were completely preserved. The extraction of unique dimension boundaries via the drop duplicates operation properly normalized the dataset, while the monotonically increasing id function generated reliable, conflict-free surrogate primary keys without requiring an active database sequence coordinator.

5.2 Alignment with Business Objectives

The ultimate objective of this project was to transform massive, unorganized education infrastructure data into an accessible format that enables stakeholders to perform rapid business intelligence and data analytics.

The pipeline's success in aligning with these goals is demonstrated via the validation script (verify.py):

1. **Aggregating Trends Across Time:** Prior to the ETL pipeline, calculating the total number of basic education enrollments per year required scanning 2.2 GB of raw text across multiple files. Post-ETL, running a simple SUM(**qt_mat_bas)** GROUP BY **nu_ano_censo** query execution completes instantly because the data is pre-sorted, typed, and isolated within the optimized fato_matriculas table.
2. **Cross-Dimensional Infrastructure Analysis:** Stakeholders can cleanly evaluate crucial infrastructure gaps such as analyzing the exact count of schools functioning with or without internet access because the analytical database utilizes indexed integer joins (**f.id_dim_in_internet = i.id**) rather than heavy text-string filters, downstream

BI dashboards like Metabase can render visualization charts interactively with sub-second latency.

5.3 System Limitations

While the Apache Spark ETL pipeline successfully builds a structured data warehouse schema, several architectural limitations have been identified:

1. **Single-Node Resource Constraints:** The local deployment configuration relies on a hardcoded standalone master setup with allocated local driver memory .This creates a vertical scaling ceiling; if the census data scale expands to include broader institutional datasets, the local host machine will face severe disk-paging bottlenecks and potential OutOfMemory (OOM) exceptions .
2. **Local Networking Dependencies and Connection Failure Risks:** As shown during execution trials in Untitled.ipynb, the target ingestion database relies on a volatile local networking stack (jdbc:postgresql://localhost:5432/). If the database service container or service daemon experiences connection blockages or is not active, the entire Spark writing stage fails instantly with a Connection refused error, lacking any automated retry mechanisms or fault-tolerant buffering.
3. **Destructive Database Writing (Lack of Incremental Processing):** The fact table generation currently operates via .mode("overwrite"). This represents a destructive load mechanism that drops the existing target table and rewrites it completely. For historical analytical data warehousing, this prevents continuous data streaming or incremental updates, forcing a costly re-computation of the entire historical dataset every time new data is added.

5.4 Proposed Future Improvements

To move this local solution into an enterprise-grade, production-ready cloud platform, the following data architecture improvements are proposed:

1. **Migration to Azure Cloud Infrastructure (Databricks + ADLS Gen2):** To achieve full elastic scalability, the local file paths should be replaced with **Azure Data Lake Storage Gen2 (ADLS Gen2)** namespaces. The PySpark transformation scripts can be directly deployed into an autoscaling **Azure Databricks** cluster, decoupling compute from storage and allowing the processing of terabyte-scale data seamlessly.
2. **Orchestration via Azure Data Factory (ADF):** The manual execution of individual Python files should be replaced by an Azure Data Factory pipeline. ADF can orchestrate the workflow using triggers and send automated email alerts if an extraction failure occurs.
3. **Adopting Delta Lake Architecture for Incremental Ingestion:** Transitioning the destination database layer from standard PostgreSQL/Parquet to a Delta Lake storage framework on the cloud would introduce ACID transactions and time-travel logging. This would enable the team to implement Slowly Changing Dimensions) and merge data incrementally via UPSERT operations, completely eliminating the costly and dangerous .mode("overwrite") design bottleneck.

Reflection

1. Muhammad Afiq Danial Bin Rozaidie

From this tutorial, I was assigned the data preparation and modeling phase in a group of three, where I was responsible for designing and implementing the Star Schema using PySpark and PostgreSQL. I encountered a significant challenge when chaining multiple joins to build the Fact Table, as Spark raised ambiguous column name errors due to duplicate column references across the joined DataFrames. I resolved this by selecting only the surrogate key and join key from each dimension table before performing the join, which eliminated the conflict and deepened my understanding of how Spark manages column scope in distributed query plans. I also learned the importance of coordinating closely with my teammate who handled Data Preparation, as mismatched column naming conventions between our pipeline stages initially caused compatibility issues that we resolved through better communication and a shared naming agreement. From this tutorial, I gained hands-on experience in PySpark DataFrame operations, Star Schema design, ETL pipeline architecture, and PostgreSQL integration, all of which I consider essential skills for a career in data engineering.

2. Muhammad Mukhritz Al Iman Bin Mohd Raffi

Completing this tutorial gave me hands-on experience with data engineering principles by building a functional ETL pipeline. My main challenges were handling the massive 370-column census dataset and resolving a local database connection error on port 5432. I overcame these by using Apache Spark to compress the raw data into Parquet files and applying broadcast joins to optimize memory constraints. Splitting the project into modular Python scripts also greatly improved my individual debugging process. Finally, this project helped me understand enterprise cloud architecture, as I can now clearly see how my local PySpark code translates to Azure Databricks, my local storage to Azure Data Lake Storage Gen2, and my manual script execution to automated orchestration using Azure Data Factory.

3. Ahmad Adib Zikri bin A.Mazlam

From the tutorial, our group mastered core data engineering competencies by overcoming significant big-data memory bottlenecks and configuration hurdles. We bypassed local hardware failures by replacing raw CSV ingestion with PySpark’s lazy evaluation and optimized Parquet storage, resulting in a massive processing speedup. Furthermore, we

resolved containerized network communication conflicts within Docker to seamlessly bridge PostgreSQL, Metabase, and local Python environments. This project provided invaluable hands-on insight into building automated, scalable, and production-ready OLAP data pipelines from raw, unmanaged source data.